Insurance Policy Intelligence Agentic RAG Copilot

FDE Project — Problem Statement, Proposed Solution, and Easy-to-Understand Examples

Customer: HDFC ERGO General Insurance Company Limited
Industry: General Insurance
Use Case: Insurance Policy Intelligence and Claims Document Assistance
Project Type: Forward Deployed Engineer (FDE) — Agentic RAG Application

Disclaimer: This is a proposed portfolio project. It is not an official HDFC ERGO product, and the examples are illustrative. Actual policy wording, coverage, exclusions, and claim requirements must be verified against the applicable official policy documents.

---

1. Problem Statement

Insurance companies maintain large collections of policy documents, policy schedules, product brochures, claim forms, terms and conditions, exclusions, add-on coverage documents, and claims-processing guidelines.

Customers, insurance agents, and claims-support employees often need to search across multiple documents to understand whether a particular event is covered, which exclusions apply, what documents are required, and how to initiate a claim.

For example, a customer may ask whether hospitalization expenses are covered under a health insurance policy, whether a particular waiting period applies, or which documents must be submitted for reimbursement.

Finding the correct answer can require reviewing multiple documents and checking the exact wording of the applicable policy. Traditional keyword search may return many irrelevant passages, while a standard chatbot may incorrectly interpret coverage conditions or generate unsupported answers.

Incorrect information can lead to customer confusion, unnecessary support calls, incomplete claim submissions, and additional work for insurance operations teams.

The Core Business Challenge

Insurance stakeholders need fast and accurate access to policy information, but the company needs every answer to be grounded in the correct policy documents, clearly explain applicable conditions and exclusions, and avoid making unsupported promises about claim approval.

2. Why Simple RAG Is Not Enough

A basic Retrieval-Augmented Generation (RAG) system follows this workflow:

Question → Retrieve Documents → Generate Answer

However, insurance policy interpretation requires additional safeguards.

- **Incorrect document retrieval:** The system may retrieve a similar policy document that does not apply to the customer's specific plan.
- **Complex policy wording:** Coverage may depend on waiting periods, limits, deductibles, exclusions, add-on benefits, and other conditions.
- **Incomplete evidence:** One retrieved passage may describe coverage while another section contains a relevant exclusion.
- **Document version conflicts:** Different policy versions may contain different terms.
- **Unsupported claim decisions:** A chatbot might incorrectly suggest that a claim is approved when the available documents do not establish that conclusion.

Therefore, the system must identify the relevant policy, evaluate the available evidence, and explain uncertainty before generating a response.

3. Proposed Solution

Build an **Insurance Policy Intelligence Agentic RAG Copilot** that helps customers and authorized insurance employees find policy information and understand claim-document requirements.

The application retrieves relevant information from approved policy documents, evaluates the retrieved evidence, and generates a grounded response with document references.

If the evidence is insufficient, the system can search an approved secondary knowledge source or ask the user for missing details. It can rewrite weak search queries and retry retrieval within a configured limit.

The system provides policy information and preliminary document checks. It does not independently make final claim-approval decisions.

Proposed Workflow

1. The user submits a question about an insurance policy or claim.

2. The system identifies the question type and determines which policy information is required.
3. It retrieves relevant documents from the private vector database.
4. It checks policy identifiers, document versions, and available metadata.
5. An evidence grader evaluates whether the retrieved passages adequately support an answer.
6. The system checks for relevant exclusions, limitations, waiting periods, and conditions.
7. If the evidence is sufficient, the LLM generates a grounded response with citations.
8. If the evidence is insufficient, the system requests clarification, searches an approved source if available, or rewrites the query and retries.
9. The system returns the answer, supporting references, limitations, and decision trace.
10. Cases requiring interpretation or an actual claim decision are referred to an authorized human reviewer.

4. Easy Example #1 — Finding Policy Coverage Conditions

Customer Question:

"Does my health insurance policy cover hospitalization expenses?"

What Happens?

The system retrieves the applicable policy wording and, where available, the customer's policy schedule.

It identifies relevant coverage provisions and checks whether any applicable conditions, exclusions, limits, or waiting periods affect the answer.

The system then explains what the documents state and identifies any missing information needed to assess the customer's particular situation.

Expected Workflow

User Question → Policy Identification → Private Document Retrieval → Evidence Grade: GOOD → Check Conditions and Exclusions → Grounded Answer with Citations

Easy Explanation

The AI does not simply answer "yes" or "no." It searches the applicable policy documents, checks the relevant conditions, and explains what the documents actually support.

Business Benefit: Customers can understand policy wording more quickly and may need fewer routine clarification calls.

5. Easy Example #2 — Checking Required Claim Documents

Customer Question:

"What documents should I prepare for a reimbursement claim?"

What Happens?

The system retrieves the applicable policy's claim instructions and relevant reimbursement checklist.

Depending on the policy and type of claim, the retrieved documents may mention items such as claim forms, hospital bills, discharge summaries, prescriptions, diagnostic reports, and payment receipts.

The system generates a checklist based on the retrieved documents and indicates when requirements depend on the claim circumstances.

It does not assume that the same checklist applies to every product or claim.

Expected Workflow

User Question → Identify Policy and Claim Type → Retrieve Claim Instructions → Grade Evidence → Generate Document Checklist → Cite Sources

Easy Explanation

The AI finds the correct claim instructions and turns them into a practical checklist, while making clear that additional documents may be requested.

Business Benefit: Better-prepared claim submissions may reduce avoidable back-and-forth communication.

6. Easy Example #3 — Conflicting or Insufficient Policy Evidence

Customer Question:

"Will my claim definitely be approved if I submit these hospital bills?"

What Happens?

The system searches the applicable policy documents and retrieves the relevant claim-submission requirements.

However, the available documents may not establish whether the particular claim satisfies every coverage condition or whether all required evidence has been submitted.

The evidence grader identifies the limitation. The system explains which requirements can be verified from the documents and what additional information may be necessary.

If an actual claim decision is needed, the case is referred to the authorized claims team.

Expected Workflow

User Question → Retrieve Policy and Claim Rules → Grade Evidence: INSUFFICIENT → Identify Missing Information → Explain Limitations → Human Review if Required

Easy Explanation

The AI does not promise claim approval merely because the customer has bills. It distinguishes between explaining policy requirements and making an official claim decision.

Business Benefit: Clearer communication and fewer unsupported assurances about coverage or claim outcomes.

7. What Makes This Agentic RAG?

Traditional RAG generally follows a fixed workflow:

Question → Retrieve → Generate

Agentic RAG introduces conditional decisions based on the retrieved evidence.

Normal RAG	Agentic RAG in This Project
Retrieves documents and generates an answer	Retrieves, grades evidence, and retries when necessary
May retrieve the wrong policy version	Uses policy identifiers and document metadata to improve relevance
May overlook exclusions	Explicitly checks for relevant conditions and exclusions
Limited handling of missing information	Requests clarification or acknowledges insufficient evidence

Generates a response after retrieval	Generates grounded answers after evidence evaluation
Limited transparency	Returns supporting citations and a workflow trace
May overstate claim certainty	Escalates decisions that require authorized human review

The system is agentic because it decides what action to take next based on the question, policy context, and quality of available evidence.

8. Proposed Technology Stack

- **LangGraph:** Controls the agent workflow, routing, evidence evaluation, and retries.
- **Pinecone:** Stores and retrieves vector representations of approved insurance documents.
- **HuggingFace Embeddings:** Converts policy documents and user questions into vectors for semantic search.
- **Groq LLM:** Supports question classification, evidence grading, query rewriting, and grounded response generation.
- **FastAPI:** Exposes chat, document-ingestion, and health-check APIs.
- **HTML, CSS, and JavaScript:** Provide a user interface for questions, answers, citations, and decision traces.
- **SQLite:** Stores basic audit records and workflow decisions.
- **Document Ingestion Pipeline:** Loads and processes PDF, TXT, Markdown, and DOCX files.
- **Docker:** Supports reproducible application packaging and deployment.

Important Data Requirements

The system should index approved policy wording, product brochures, policy schedules where authorized, claim instructions, and relevant supporting documents.

Each document should include metadata where available, such as:

- Product and policy identifier.
- Policy version or effective date.
- Document type.
- Section or clause number.
- Source document name.
- Page number.
- Applicable jurisdiction or product variant, if relevant.

Access controls must prevent one customer's private policy documents from being exposed to another customer.

9. Expected Business Value

The proposed system aims to:

- Help customers and employees find policy information faster.
- Reduce repetitive policy-related support questions.
- Improve access to relevant coverage conditions and exclusions.
- Provide traceable answers supported by policy documents.
- Reduce unsupported statements about coverage and claim approval.
- Turn lengthy claim-document requirements into practical checklists.
- Identify missing information and cases requiring human review.
- Make policy-document updates easier to incorporate through controlled ingestion.
- Improve transparency through source references and workflow traces.

These benefits should be measured during a pilot using answer accuracy, citation correctness, retrieval quality, response time, escalation rates, and user feedback.

10. FDE Perspective

A Forward Deployed Engineer does more than build a chatbot. The FDE understands the customer's operational workflow and delivers an application that integrates relevant data, business rules, APIs, user interfaces, security, evaluation, and deployment.

This project demonstrates the following stages:

1. Understand the insurance policy-support workflow.
2. Identify document sources and access requirements.
3. Build a policy-document ingestion pipeline.
4. Develop the Agentic RAG workflow.
5. Implement retrieval evaluation and policy-version filtering.
6. Build APIs and a user interface.
7. Add citations, audit logging, and access controls.
8. Evaluate responses against verified policy examples.
9. Package and deploy the application.
10. Monitor performance and improve it using reviewer feedback.

End-to-End FDE Lifecycle

Customer Problem → Requirements Discovery → Policy Data Integration → Agentic RAG Development → API Development → User Interface → Security and Evaluation → Deployment → Monitoring → Improvement

11. Suggested Demonstration Scenarios

Scenario A — Policy Coverage Question

Question: "Does my health insurance policy cover hospitalization expenses?"

Expected Behavior: Retrieve the applicable policy wording, check relevant conditions and exclusions, and answer with citations.

Scenario B — Claim Document Checklist

Question: "What documents should I prepare for a reimbursement claim?"

Expected Behavior: Retrieve the relevant claim instructions and generate a source-supported checklist.

Scenario C — Waiting Period or Exclusion

Question: "Is this particular treatment covered immediately after I purchase the policy?"

Expected Behavior: Identify the applicable policy and retrieve the relevant waiting-period and exclusion clauses. If policy-specific details are missing, ask for clarification.

Scenario D — Conflicting or Outdated Documents

Question: "One brochure says this benefit is included, but another document appears to exclude it. Which applies?"

Expected Behavior: Compare document versions and applicability metadata, explain the conflict, and refer unresolved interpretation to an authorized reviewer.

Scenario E — Claim Approval Uncertainty

Question: "Will my claim definitely be approved?"

Expected Behavior: Explain what can be verified from the supplied documents, identify missing information, and avoid presenting a final claim decision without authorized review.

Scenario F — Direct Conversation

Question: "Hello!"

Expected Behavior: Return a simple greeting without unnecessary document retrieval.

12. Security, Governance, and Limitations

Because insurance documents can contain personal and financial information, the application should incorporate appropriate safeguards.

- Authenticate users and enforce role-based authorization.
- Restrict access to customer-specific policy information.
- Protect API keys and uploaded files.
- Validate document types and file sizes.
- Maintain appropriate audit records without unnecessarily storing sensitive information.
- Treat external search results as untrusted until verified.
- Display citations that identify the relevant source document and clause or page when available.
- Do not treat retrieved text as instructions to override system security rules.
- Require authorized human review for disputed interpretations and actual claim decisions.

The application is an informational assistant, not a replacement for official policy wording, professional advice, or the insurer's authorized claims process.

13. Conclusion

The Insurance Policy Intelligence Agentic RAG Copilot is a practical enterprise AI portfolio project focused on a real operational challenge: helping users understand complex insurance documents without relying on unsupported chatbot responses.

By combining document retrieval, evidence evaluation, policy metadata, conditional routing, query rewriting, source citations, and human escalation, the project demonstrates how Agentic RAG can be applied to a document-heavy business workflow.

It provides a foundation for evaluating how AI can improve policy-information access, support claim-document preparation, and make insurance customer service more transparent.

Final takeaway:

Customer Problem → Understand Policy Workflows → Integrate Approved Documents → Retrieve Relevant Evidence → Evaluate Conditions and Exclusions → Generate Cited Answers → Escalate When Necessary → Test → Deploy → Monitor and Improve

Project status: Proposed portfolio design. No affiliation with, endorsement by, or production deployment at HDFC ERGO is implied.